WEEK 5

Layouts, Material 3, Themes & Lists

Daily study material with detailed examples

Android Development with Kotlin | Prepared for **Anjana**

Last week you built single screens from basic pieces. This week you'll master **layouts** (arranging anything a designer draws), use more **Material 3 components**, make the app follow a **theme** (including dark mode), and show long **scrollable lists** efficiently. At the end you'll build a **Contact List app**.

Day	Topic	You will be able to...
Day 1	Layouts in depth	Use arrangement, alignment, weight and Box confidently
Day 2	Material 3 components & Scaffold	Build screens with top bars, FABs, chips and dividers
Day 3	Themes, colours & fonts	Customise the theme, support dark mode, add fonts
Day 4	Lists with LazyColumn	Show long lists of data efficiently
Day 5	LazyRow, grids & list patterns	Build horizontal lists, grids, headers and empty states
Day 6	Weekly project	Build a Contact List app

Tip: Every Compose example was **compile-checked**. Imports are left out, so press **Alt + Enter** on red words. Use a new practice project, **Week5Practice**.

WEEK 5 • DAY 1

Layouts in Depth

Today's goal: Arrange elements exactly as designed using Column, Row, Box, arrangement, alignment and weight.

1. The three layouts

Layout	Children are placed...	Main direction	Cross direction
Column	top to bottom	vertical (verticalArrangement)	horizontal (horizontalAlignment)
Row	left to right	horizontal (horizontalArrangement)	vertical (verticalAlignment)
Box	on top of each other	-	contentAlignment / Modifier.align()

Tip: Easy rule: **arrangement** is along the layout's own direction (how items are **spread out**). **Alignment** is across it (how items **line up**).

2. Arrangement: spreading items out

All the horizontal arrangements

```
@Composable
fun ArrangementRow(label: String, arrangement: Arrangement.Horizontal) {
    Column {
        Text(label, style = MaterialTheme.typography.labelMedium)
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .background(MaterialTheme.colorScheme.surfaceVariant),
            horizontalArrangement = arrangement
        ) {
            repeat(3) { index ->
                Box(
                    modifier = Modifier
                        .size(40.dp)
                        .background(MaterialTheme.colorScheme.primary),
                    contentAlignment = Alignment.Center
                ) {
                    Text("${index + 1}", color = MaterialTheme.colorScheme.onPrimary)
                }
            }
        }
    }
}

@Composable
fun ArrangementDemo() {
    Column(modifier = Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(12.dp)) {
        ArrangementRow("Start", Arrangement.Start)
        ArrangementRow("Center", Arrangement.Center)
        ArrangementRow("End", Arrangement.End)
        ArrangementRow("SpaceBetween", Arrangement.SpaceBetween)
        ArrangementRow("SpaceAround", Arrangement.SpaceAround)
        ArrangementRow("SpaceEvenly", Arrangement.SpaceEvenly)
        ArrangementRow("spacedBy(8.dp)", Arrangement.spacedBy(8.dp))
    }
}
```

Sketch: Result

```
Start          |[1][2][3]                      |
Center         |          [1][2][3]            |
End            |                     [1][2][3]|
SpaceBetween   |[1]           [2]          [3]|
SpaceAround    |  [1]        [2]        [3]   |   (half-size gaps at the edges)
SpaceEvenly    |    [1]      [2]      [3]     |   (equal gaps everywhere)
spacedBy(8dp)  |[1] [2] [3]                    |
```

Notice how **ArrangementRow** takes the arrangement as a parameter, so one composable shows all seven options. Writing a small helper like this is a great way to experiment.

3. Alignment: lining items up

Vertical alignment in a Row

```
@Composable
fun AlignmentDemo() {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .height(100.dp)
            .background(MaterialTheme.colorScheme.surfaceVariant),
        horizontalArrangement = Arrangement.SpaceEvenly
    ) {
        Text("Top", modifier = Modifier.align(Alignment.Top))
        Text("Center", modifier = Modifier.align(Alignment.CenterVertically))
        Text("Bottom", modifier = Modifier.align(Alignment.Bottom))
    }
}
```

verticalAlignment on the Row aligns **all** children. **Modifier.align()** on one child overrides it for that child only.

4. weight: sharing space

weight in action

```
@Composable
fun WeightDemo() {
    Column(modifier = Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(16.dp)) {
        // Equal thirds
        Row(modifier = Modifier.fillMaxWidth().height(40.dp)) {
            Box(Modifier.weight(1f).fillMaxHeight().background(MaterialTheme.colorScheme.primary))
            Box(Modifier.weight(1f).fillMaxHeight().background(MaterialTheme.colorScheme.secondary))
            Box(Modifier.weight(1f).fillMaxHeight().background(MaterialTheme.colorScheme.tertiary))
        }
        // 2 : 1
        Row(modifier = Modifier.fillMaxWidth().height(40.dp)) {
            Box(Modifier.weight(2f).fillMaxHeight().background(MaterialTheme.colorScheme.primary))
            Box(Modifier.weight(1f).fillMaxHeight().background(MaterialTheme.colorScheme.secondary))
        }
        // Text fills the gap, the icon stays at the end
        Row(verticalAlignment = Alignment.CenterVertically) {
            Icon(Icons.Default.Notifications, contentDescription = null)
            Spacer(Modifier.width(16.dp))
            Text("Push notifications", modifier = Modifier.weight(1f))
            Icon(Icons.AutoMirrored.Filled.KeyboardArrowRight, contentDescription = null)
        }
    }
}
```

Pattern	How
Equal-width buttons	Give each one Modifier.weight(1f)
Push something to the end of a Row	Give the element before it Modifier.weight(1f)
Push something to the bottom of a screen	In a Column: Spacer(Modifier.weight(1f)) above it
Proportions (2:1)	weight(2f) and weight(1f)

5. Box: layering

Content on top of content

```
@Composable
fun BoxDemo() {
    Box(
        modifier = Modifier
            .padding(16.dp)
            .fillMaxWidth()
            .height(160.dp)
            .clip(RoundedCornerShape(16.dp))
            .background(MaterialTheme.colorScheme.primaryContainer)
    ) {
        Text("TopStart", modifier = Modifier.align(Alignment.TopStart).padding(12.dp))
        Text("TopEnd", modifier = Modifier.align(Alignment.TopEnd).padding(12.dp))
        Text("Center", modifier = Modifier.align(Alignment.Center))
        Text("BottomStart", modifier = Modifier.align(Alignment.BottomStart).padding(12.dp))
        FloatingActionButton(
            onClick = { },
            modifier = Modifier.align(Alignment.BottomEnd).padding(12.dp)
        ) {
            Icon(Icons.Default.Add, contentDescription = "Add")
        }
    }
}
```

Box children are drawn in order, so **later children appear on top**. Use Box for badges on icons, text over images, and floating buttons in a corner.

6. Real example: a product item

ProductItem

```
@Composable
fun ProductItem(name: String, price: Int, rating: Double, modifier: Modifier = Modifier) {
    Row(
        modifier = modifier.fillMaxWidth().padding(16.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Box(
            modifier = Modifier
                .size(72.dp)
                .clip(RoundedCornerShape(12.dp))
                .background(MaterialTheme.colorScheme.surfaceVariant),
            contentAlignment = Alignment.Center
        ) {
            Icon(Icons.Default.ShoppingCart, contentDescription = null)
        }
        Spacer(Modifier.width(16.dp))
        Column(modifier = Modifier.weight(1f)) {
            Text(name, style = MaterialTheme.typography.titleMedium, maxLines = 1,
                overflow = TextOverflow.Ellipsis)
            Row(verticalAlignment = Alignment.CenterVertically) {
                Icon(Icons.Default.Star, contentDescription = null, modifier = Modifier.size(16.dp),
                    tint = MaterialTheme.colorScheme.primary)
                Text(" $rating", style = MaterialTheme.typography.bodySmall)
            }
            Text("Rs $price", style = MaterialTheme.typography.titleSmall,
                color = MaterialTheme.colorScheme.primary)
        }
        FilledTonalButton(onClick = { }) { Text("Add") }
    }
}
```

Tip: Build any design in 3 steps: (1) draw boxes on the design: vertical = Column, horizontal = Row, layered = Box; (2) find the part that fills the remaining space and give it weight(1f); (3) add spacing and styles last.

Practice Exercises

1. Recreate this layout: a row with a 48.dp avatar circle, a name/time Column with weight(1f), and an unread badge (a small circle with a number) at the end.

2. Build a **LoginLayout**: a title at the top, two OutlinedButtons in the middle, and a full-width Button pushed to the **bottom** with Spacer(Modifier.weight(1f)).

3. Build a 2x2 grid of equal coloured squares using two Rows with weight(1f) children inside a Column.

4. Build a banner Box with text at the BottomStart and a close IconButton at the TopEnd.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. In a Column, which parameter spreads items vertically?

Q2. What is the difference between SpaceBetween and SpaceEvenly?

Q3. How do you push a button to the bottom of a Column?

Q4. In a Box, which child is drawn on top?

Q5. How do you align just one child differently in a Row?

WEEK 5 • DAY 2

Material 3 Components & Scaffold

Today's goal: Use common Material 3 components and build complete screens with Scaffold.

1. What is Material 3?

Material Design 3 (Material You) is Google's design system: a set of ready-made, accessible components with consistent colours, shapes and typography. Compose's **material3** library gives you all of them.

2. Scaffold: the screen frame

A complete screen with Scaffold

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun NotesScreen(onAddNote: () -> Unit) {
    Scaffold(
        topBar = {
            CenterAlignedTopAppBar(
                title = { Text("My Notes") },
                navigationIcon = {
                    IconButton(onClick = { }) {
                        Icon(Icons.Default.Menu, contentDescription = "Menu")
                    }
                },
                actions = {
                    IconButton(onClick = { }) {
                        Icon(Icons.Default.Search, contentDescription = "Search")
                    }
                }
            )
        },
        floatingActionButton = {
            ExtendedFloatingActionButton(
                onClick = onAddNote,
                icon = { Icon(Icons.Default.Add, contentDescription = null) },
                text = { Text("New note") }
            )
        }
    ) { innerPadding ->
        Column(
            modifier = Modifier
                .padding(innerPadding)
                .fillMaxSize()
                .padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            Text("Today", style = MaterialTheme.typography.titleMedium)
            OutlinedCard(modifier = Modifier.fillMaxWidth()) {
                Text("Buy vegetables", modifier = Modifier.padding(16.dp))
            }
            OutlinedCard(modifier = Modifier.fillMaxWidth()) {
                Text("Finish Week 5 lessons", modifier = Modifier.padding(16.dp))
            }
        }
    }
}
```

Careful: Always apply **innerPadding** to your content, or it will be hidden behind the top bar.
@OptIn(ExperimentalMaterial3Api::class) is required for the top app bars. They are safe to use.

3. Bottom navigation bar

NavigationBar

```
@Composable
fun AppBottomBar(selectedTab: Int, onTabSelected: (Int) -> Unit) {
    NavigationBar {
        NavigationBarItem(
            selected = selectedTab == 0,
            onClick = { onTabSelected(0) },
            icon = { Icon(Icons.Default.Home, contentDescription = null) },
            label = { Text("Home") }
        )
        NavigationBarItem(
            selected = selectedTab == 1,
            onClick = { onTabSelected(1) },
            icon = { Icon(Icons.Default.Favorite, contentDescription = null) },
            label = { Text("Favourites") }
        )
        NavigationBarItem(
            selected = selectedTab == 2,
            onClick = { onTabSelected(2) },
            icon = { Icon(Icons.Default.Person, contentDescription = null) },
            label = { Text("Profile") }
        )
    }
}
```

Pass it to Scaffold with **bottomBar = { AppBottomBar(...) }**. Remembering which tab is selected needs **state** (Week 6).

4. Chips

Assist, filter and suggestion chips

```
@Composable
fun ChipExamples() {
    Column(modifier = Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(8.dp)) {
        AssistChip(
            onClick = { },
            label = { Text("Add to calendar") },
            leadingIcon = { Icon(Icons.Default.DateRange, contentDescription = null) }
        )
        Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            FilterChip(selected = true, onClick = { }, label = { Text("Veg") })
            FilterChip(selected = false, onClick = { }, label = { Text("Non-veg") })
        }
        SuggestionChip(onClick = { }, label = { Text("Show nearby") })
        InputChip(
            selected = false,
            onClick = { },
            label = { Text("Kochi") },
            trailingIcon = { Icon(Icons.Default.Close, contentDescription = "Remove") }
        )
    }
}
```

Chip	Use
AssistChip	A quick action related to the content
FilterChip	Turning a filter on or off (selected / not selected)
SuggestionChip	A suggested reply or search

Chip	Use
InputChip	Something the user entered (a tag, a recipient) that can be removed

5. Dividers, badges and progress indicators

Small but common components

```
@Composable
fun SmallComponents() {
    Column(modifier = Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(16.dp)) {
        Text("Above the divider")
        HorizontalDivider()
        Text("Below the divider")

        BadgedBox(badge = { Badge { Text("4") } }) {
            Icon(Icons.Default.Notifications, contentDescription = "4 new notifications")
        }

        LinearProgressIndicator(progress = { 0.6f }, modifier = Modifier.fillMaxWidth())
        CircularProgressIndicator()
    }
}
```

LinearProgressIndicator(progress = { 0.6f }) shows 60% progress. Without **progress**, both indicators animate endlessly ("loading, please wait").

6. Text fields (a first look)

TextField appearance

```
@Composable
fun TextFieldLooks() {
    Column(modifier = Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(12.dp)) {
        OutlinedTextField(
            value = "anjana@example.com",
            onValueChange = { },
            label = { Text("Email") },
            leadingIcon = { Icon(Icons.Default.Email, contentDescription = null) },
            modifier = Modifier.fillMaxWidth()
        )
        TextField(
            value = "",
            onValueChange = { },
            label = { Text("Search") },
            leadingIcon = { Icon(Icons.Default.Search, contentDescription = null) },
            modifier = Modifier.fillMaxWidth()
        )
    }
}
```

These fields show fixed text and can't be edited yet: typing needs **state**, which you'll learn in **Week 6**.

Practice Exercises

1. Build a **ProfileScreen** with a TopAppBar (title "Profile", back arrow navigation icon, Edit action) and a Column of info cards.

2. Add the AppBottomBar to the NotesScreen Scaffold, with the Home tab selected.

3. Build a row of FilterChips for categories: All (selected), Work, Personal, Shopping.

4. Build a "Download" card with a title, a LinearProgressIndicator at 35% and a "35%" label at the end of the row.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What does Scaffold do?

Q2. Why must you use innerPadding?

Q3. Which chip should you use for a filter that can be on or off?

Q4. How do you show a number badge on an icon?

Q5. Why can't you type in the text fields yet?

WEEK 5 • DAY 3

Themes, Colours & Fonts

Today's goal: Understand the theme files, customise colours, typography and shapes, and support dark mode.

1. The three theme files

File (ui/theme)	Contains
Color.kt	Your colour values, e.g. val Green40 = Color(0xFF1E7A4C)
Theme.kt	The light and dark colour schemes, and the theme composable (e.g. Week5PracticeTheme)
Type.kt	The typography: fonts and text styles

2. Defining colours and the theme

Color.kt (Android only)

```
val Green40 = Color(0xFF1E7A4C)
val Green80 = Color(0xFF4CC68A)
val Purple40 = Color(0xFF6B4FD8)
val Purple80 = Color(0xFFA994FF)
```

Theme.kt (Android only)

```
private val LightColorScheme = lightColorScheme(
    primary = Green40,
    secondary = Purple40,
    // any colour you don't set keeps the Material default
)

private val DarkColorScheme = darkColorScheme(
    primary = Green80,
    secondary = Purple80,
)

@Composable
fun Week5PracticeTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = false,           // true: use the phone wallpaper colours (Android 12+)
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context) else dynamicLightColorScheme(context)
        }
        darkTheme -> DarkColorScheme
        else -> LightColorScheme
    }
    MaterialTheme(colorScheme = colorScheme, typography = Typography, content = content)
}
```

isSystemInDarkTheme() follows the phone's dark mode setting. **Dynamic color** (Android 12 and newer) builds a colour scheme from the user's wallpaper. Many company apps turn it **off** to keep their brand colours.

3. Using colours correctly

Theme colours in components

```
@Composable
fun ColorRolesDemo() {
    Column(modifier = Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(8.dp)) {
        ColorRole("primary", MaterialTheme.colorScheme.primary, MaterialTheme.colorScheme.onPrimary)
        ColorRole("primaryContainer", MaterialTheme.colorScheme.primaryContainer,
            MaterialTheme.colorScheme.onPrimaryContainer)
        ColorRole("secondaryContainer", MaterialTheme.colorScheme.secondaryContainer,
            MaterialTheme.colorScheme.onSecondaryContainer)
        ColorRole("surfaceVariant", MaterialTheme.colorScheme.surfaceVariant,
            MaterialTheme.colorScheme.onSurfaceVariant)
        ColorRole("error", MaterialTheme.colorScheme.error, MaterialTheme.colorScheme.onError)
    }
}

@Composable
fun ColorRole(name: String, background: Color, content: Color) {
    Text(
        text = name,
        color = content,
        modifier = Modifier
            .fillMaxWidth()
            .background(background, RoundedCornerShape(8.dp))
            .padding(12.dp)
    )
}
```

The golden rule: put **onX** content on an **X** background (onPrimary text on primary). Then text is readable in light **and** dark mode.

BAD: fixed colours don't adapt to dark mode

```
Text("Title", color = Color.Black)                // invisible on a dark background
Card(colors = CardDefaults.cardColors(containerColor = Color.White))   // always white
```

4. Typography and custom fonts

- Download a font (for example **Poppins** from fonts.google.com) and copy the .ttf files into **res/font/** (create the folder: right-click res → New → Android Resource Directory → font).
- File names must be lowercase: poppins_regular.ttf, poppins_bold.ttf.
- Define a FontFamily and use it in Type.kt.

Type.kt with a custom font (Android only)

```
val Poppins = FontFamily(
    Font(R.font.poppins_regular, FontWeight.Normal),
    Font(R.font.poppins_bold, FontWeight.Bold)
)

val Typography = Typography(
    titleLarge = TextStyle(fontFamily = Poppins, fontWeight = FontWeight.Bold, fontSize = 22.sp),
    bodyLarge = TextStyle(fontFamily = Poppins, fontWeight = FontWeight.Normal, fontSize = 16.sp,
        lineHeight = 24.sp)
    // styles you don't set keep the defaults
)
```

5. Shapes

Using theme shapes

```
@Composable
fun ShapeTokens() {
    Row(modifier = Modifier.padding(16.dp), horizontalArrangement = Arrangement.spacedBy(12.dp)) {
        Box(Modifier.size(56.dp).background(MaterialTheme.colorScheme.primary,
            MaterialTheme.shapes.small))
        Box(Modifier.size(56.dp).background(MaterialTheme.colorScheme.primary,
            MaterialTheme.shapes.medium))
        Box(Modifier.size(56.dp).background(MaterialTheme.colorScheme.primary,
            MaterialTheme.shapes.large))
        Box(Modifier.size(56.dp).background(MaterialTheme.colorScheme.primary,
            MaterialTheme.shapes.extraLarge))
    }
}
```

6. Previewing light and dark

A composable that works in both themes

```
@Composable
fun WelcomeBanner(name: String) {
    Surface(
        color = MaterialTheme.colorScheme.primaryContainer,
        shape = MaterialTheme.shapes.large,
        modifier = Modifier.fillMaxWidth().padding(16.dp)
    ) {
        Column(modifier = Modifier.padding(20.dp)) {
            Text("Hello, $name", style = MaterialTheme.typography.headlineSmall)
            Text("Ready to learn something new today?",
                style = MaterialTheme.typography.bodyMedium)
        }
    }
}

@Composable
fun WelcomeBannerBothThemes() {
    Column {
        MaterialTheme(colorScheme = lightColorScheme()) {
            WelcomeBanner(name = "Anjana")
        }
        MaterialTheme(colorScheme = darkColorScheme()) {
            Surface { WelcomeBanner(name = "Anjana") }
        }
    }
}
```

The usual way: a dark-mode preview (Android only)

```
@Preview(name = "Light", showBackground = true)
@Preview(name = "Dark", showBackground = true, uiMode = Configuration.UI_MODE_NIGHT_YES)
@Composable
fun WelcomeBannerPreview() {
    Week5PracticeTheme {
        WelcomeBanner(name = "Anjana")
    }
}
```

Tip: Test dark mode on your device too: **Settings** → **Display** → **Dark theme**. Look for text you can't read, which almost always means a fixed colour somewhere.

Practice Exercises

1. Change your practice app's primary colour in Theme.kt to a colour you like, for both light and dark schemes. Run the app.

2. Find any fixed colours (Color.Black, Color(0xFF...)) in your Week 4 Business Card and replace them with theme colours.

3. Add a Google Font to res/font and use it for titleLarge and bodyLarge in Type.kt. (Android only.)

4. Write **InfoChip(text)** using secondaryContainer / onSecondaryContainer and MaterialTheme.shapes.small, then show it in both themes.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Which file defines the light and dark colour schemes?

Q2. What does isSystemInDarkTheme() do?

Q3. What colour should text on a primaryContainer background be?

Q4. Where do custom font files go?

Q5. What is dynamic color?

WEEK 5 • DAY 4

Lists with LazyColumn

Today's goal: Display long lists of data efficiently with LazyColumn, keys, item composables and click callbacks.

1. Why not Column + a loop?

A Column with a for loop creates **every** item at once. With 1,000 contacts that's slow, and it doesn't scroll. **LazyColumn** only creates the items **visible on screen** and reuses them as you scroll.

	Column + for loop	LazyColumn
Creates	All items, immediately	Only the visible items
Scrolls	Only with verticalScroll	Yes, built in
Good for	A few fixed items	Lists of any length, especially data from a server or database

2. Step 1: the data

A data class and sample data

```
data class Contact(
    val id: Int,
    val name: String,
    val phone: String,
    val isFavorite: Boolean = false
)

val sampleContacts = listOf(
    Contact(1, "Anjana Menon", "98470 11111", isFavorite = true),
    Contact(2, "Rahul Nair", "90000 22222"),
    Contact(3, "Meera Thomas", "94470 33333", isFavorite = true),
    Contact(4, "John Mathew", "98950 44444"),
    Contact(5, "Fathima Rasheed", "97450 55555"),
    Contact(6, "Arun Kumar", "96330 66666"),
    Contact(7, "Divya Pillai", "99610 77777"),
    Contact(8, "Sanjay Varma", "95390 88888")
)
```

3. Step 2: one item

The item composable

```
@Composable
fun ContactItem(contact: Contact, modifier: Modifier = Modifier) {
    Row(
        modifier = modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp, vertical = 12.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Column(modifier = Modifier.weight(1f)) {
            Text(contact.name, style = MaterialTheme.typography.bodyLarge)
            Text(contact.phone, style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onSurfaceVariant)
        }
        if (contact.isFavorite) {
            Icon(Icons.Default.Star, contentDescription = "Favourite",
                tint = MaterialTheme.colorScheme.primary)
        }
    }
}
```

4. Step 3: the list

LazyColumn with items

```
@Composable
fun ContactList(contacts: List<Contact>, modifier: Modifier = Modifier) {
    LazyColumn(modifier = modifier) {
        items(items = contacts, key = { contact -> contact.id }) { contact ->
            ContactItem(contact = contact)
            HorizontalDivider()
        }
    }
}

@Preview(showBackground = true)
@Composable
fun ContactListPreview() {
    ContactList(contacts = sampleContacts)
}
```

Code	Meaning
LazyColumn { }	A vertical, scrollable, lazy list. Inside the braces is a **LazyListScope**, not a normal composable
items(items = contacts) { contact -> }	One item per element in the list. The lambda describes the UI for one contact
key = { contact -> contact.id }	A **unique, stable** ID per item. Keeps scrolling and animations correct when the list changes

Careful: Two items with the same key make the app **crash**. Use a real ID from your data, never the position in the list or a name that might repeat.

5. Clicks: let the caller decide

Item clicks with a callback

```
@Composable
fun ClickableContactList(
    contacts: List<Contact>,
    onContactClick: (Contact) -> Unit,
    modifier: Modifier = Modifier
) {
    LazyColumn(modifier = modifier) {
        items(contacts, key = { it.id }) { contact ->
            Text(
                text = contact.name,
                modifier = Modifier
                    .fillMaxWidth()
                    .clickable { onContactClick(contact) }
                    .padding(16.dp)
            )
        }
    }
}
```

The list doesn't know what should happen on a click (open details? call the number?). It just reports which contact was tapped through **onContactClick**. This makes the list reusable. You'll connect it to navigation in Week 7.

6. Spacing and padding in lists

contentPadding and spacedBy

```
@Composable
fun CardList(contacts: List<Contact>) {
    LazyColumn(
        contentPadding = PaddingValues(16.dp),                // space around the whole list
        verticalArrangement = Arrangement.spacedBy(12.dp)     // space between items
    ) {
        items(contacts, key = { it.id }) { contact ->
            ElevatedCard(modifier = Modifier.fillMaxWidth()) {
                Column(modifier = Modifier.padding(16.dp)) {
                    Text(contact.name, style = MaterialTheme.typography.titleMedium)
                    Text(contact.phone, style = MaterialTheme.typography.bodyMedium)
                }
            }
        }
    }
}
```

Tip: Use **contentPadding** rather than Modifier.padding on a LazyColumn: items then scroll right to the screen edge instead of being cut off inside the padding.

7. itemsIndexed: when you need the position

Numbered list

```
@Composable
fun RankingList(names: List<String>) {
    LazyColumn {
        itemsIndexed(names, key = { _, name -> name }) { index, name ->
            Text("${index + 1}. $name", modifier = Modifier.padding(16.dp))
        }
    }
}
```

Practice Exercises

1. Create a data class **Message(id, sender, preview, time)** with 10 sample messages, and show them in a LazyColumn with dividers.

2. Make each message item show the sender in bold if it's unread (add **isUnread: Boolean**).

3. Show the contacts as cards with contentPadding 16.dp and 8.dp spacing, and add an onContactClick callback.

4. Build a numbered "Top 10 movies" list using itemsIndexed.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Why use LazyColumn instead of Column with a loop?

Q2. What is the key used for?

Q3. What happens if two items have the same key?

Q4. Why does the list take an onContactClick parameter?

Q5. What is the difference between contentPadding and Modifier.padding on a LazyColumn?

WEEK 5 • DAY 5

LazyRow, Grids & List Patterns

Today's goal: Build horizontal lists and grids, add headers and sections, and handle empty lists.

1. LazyRow: horizontal lists

Stories-style row

```
@Composable
fun StoryRow(names: List<String>) {
    LazyRow(
        contentPadding = PaddingValues(horizontal = 16.dp),
        horizontalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        items(names, key = { it }) { name ->
            Column(horizontalAlignment = Alignment.CenterHorizontally) {
                Box(
                    modifier = Modifier
                        .size(64.dp)
                        .clip(CircleShape)
                        .border(2.dp, MaterialTheme.colorScheme.primary, CircleShape)
                        .background(MaterialTheme.colorScheme.primaryContainer),
                    contentAlignment = Alignment.Center
                ) {
                    Text(name.first().toString(), style = MaterialTheme.typography.titleLarge)
                }
                Spacer(Modifier.height(4.dp))
                Text(name, style = MaterialTheme.typography.labelSmall)
            }
        }
    }
}
```

2. Grids

LazyVerticalGrid

```
data class Category(val id: Int, val name: String, val icon: ImageVector)

val categories = listOf(
    Category(1, "Home", Icons.Default.Home),
    Category(2, "Shopping", Icons.Default.ShoppingCart),
    Category(3, "Travel", Icons.Default.LocationOn),
    Category(4, "Bills", Icons.Default.DateRange),
    Category(5, "Friends", Icons.Default.Person),
    Category(6, "Settings", Icons.Default.Settings)
)

@Composable
fun CategoryGrid(items: List<Category>, onCategoryClick: (Category) -> Unit) {
    LazyVerticalGrid(
        columns = GridCells.Fixed(3),
        contentPadding = PaddingValues(16.dp),
        horizontalArrangement = Arrangement.spacedBy(12.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        items(items, key = { it.id }) { category ->
            Card(onClick = { onCategoryClick(category) }) {
                Column(
                    modifier = Modifier.fillMaxWidth().padding(16.dp),
                    horizontalAlignment = Alignment.CenterHorizontally
                ) {
                    Icon(category.icon, contentDescription = null)
                    Spacer(Modifier.height(8.dp))
                    Text(category.name, style = MaterialTheme.typography.labelLarge)
                }
            }
        }
    }
}
```

Columns	Meaning
GridCells.Fixed(3)	Always exactly 3 columns
GridCells.Adaptive(120.dp)	As many columns as fit, each at least 120.dp wide (adapts to tablets)

There's also **LazyHorizontalGrid**, and **LazyVerticalStaggeredGrid** for Pinterest-style layouts with items of different heights.

3. Headers and sections

Sections with item and items

```
@Composable
fun SectionedList(favourites: List<String>, others: List<String>) {
    LazyColumn(contentPadding = PaddingValues(vertical = 8.dp)) {
        item {
            SectionHeader("Favourites")
        }
        items(favourites, key = { "fav-$it" }) { name ->
            Text(name, modifier = Modifier.fillMaxWidth().padding(horizontal = 16.dp, vertical = 12.dp))
        }
        item {
            SectionHeader("Everyone else")
        }
        items(others, key = { "other-$it" }) { name ->
            Text(name, modifier = Modifier.fillMaxWidth().padding(horizontal = 16.dp, vertical = 12.dp))
        }
    }
}

@Composable
fun SectionHeader(title: String) {
    Text(
        text = title,
        style = MaterialTheme.typography.titleSmall,
        color = MaterialTheme.colorScheme.primary,
        modifier = Modifier.padding(start = 16.dp, top = 16.dp, bottom = 4.dp)
    )
}
```

item { } adds one element (like a header) and **items(list) { }** adds many. The key prefixes ("fav-", "other-") keep keys unique even if the same name appears in both sections.

4. A horizontal list inside a vertical list

Nested lazy lists in different directions

```
@Composable
fun HomeFeed(stories: List<String>, posts: List<String>) {
    LazyColumn {
        item {
            Text("Stories", style = MaterialTheme.typography.titleMedium,
                modifier = Modifier.padding(16.dp))
            LazyRow(
                contentPadding = PaddingValues(horizontal = 16.dp),
                horizontalArrangement = Arrangement.spacedBy(8.dp)
            ) {
                items(stories, key = { it }) { story ->
                    AssistChip(onClick = { }, label = { Text(story) })
                }
            }
        }
        items(posts, key = { it }) { post ->
            OutlinedCard(modifier = Modifier.fillMaxWidth().padding(horizontal = 16.dp, vertical = 6.dp)) {
                Text(post, modifier = Modifier.padding(16.dp))
            }
        }
    }
}
```

Careful: A LazyRow inside a LazyColumn is fine (different directions). A **LazyColumn inside another vertically scrolling** LazyColumn or Column crashes with "infinite height constraints". Use one LazyColumn with item { } blocks instead.

5. Empty states

Show a message when the list is empty

```
@Composable
fun TaskListOrEmpty(tasks: List<String>) {
    if (tasks.isEmpty()) {
        Column(
            modifier = Modifier.fillMaxSize().padding(32.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Icon(Icons.Default.CheckCircle, contentDescription = null,
                modifier = Modifier.size(72.dp), tint = MaterialTheme.colorScheme.outline)
            Spacer(Modifier.height(16.dp))
            Text("No tasks yet", style = MaterialTheme.typography.titleMedium)
            Text("Tap + to add your first task", style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onSurfaceVariant)
        }
    } else {
        LazyColumn {
            items(tasks, key = { it }) { task ->
                Text(task, modifier = Modifier.fillMaxWidth().padding(16.dp))
            }
        }
    }
}
```

Tip: Every list screen in a real app needs an **empty state**. Designers and reviewers always ask: "What does this look like with no data?"

Practice Exercises

1. Build a LazyRow of 6 "category" FilterChips with the first one selected.

2. Build a product grid with GridCells.Adaptive(150.dp): each card has a grey image placeholder Box (aspectRatio(1f)), a name and a price.

3. Build a sectioned list of countries grouped by first letter, using groupBy (Week 2) and a header item per letter.

4. Add an empty state to the Day 4 ContactList when the list is empty.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Which composable shows a horizontal lazy list?

Q2. What is the difference between GridCells.Fixed and GridCells.Adaptive?

Q3. How do you add a single header to a LazyColumn?

Q4. Why add prefixes like "fav-" to keys?

Q5. Can you put a LazyRow inside a LazyColumn?

WEEK 5 • DAY 6

Weekly Project: Contact List App

Today's goal: Build a polished Contact List app with a top bar, favourites row, alphabetical sections and an empty state.

1. The design

Sketch: What we're building

```
+--------------------------------------+
| Contacts                      (Q)    |   <- TopAppBar with search icon
+--------------------------------------+
| Favourites                           |
|  (AM)    (MT)    (AK)                |   <- LazyRow of favourite avatars
|  Anjana  Meera   Arun                |
| ------------------------------------ |
| A                                    |   <- section header per letter
| (AK) Arun Kumar          (call)      |
| (AM) Anjana Menon        (call)      |
| D                                    |
| (DP) Divya Pillai        (call)      |
| ...                                  |
|                              [ + ]   |   <- FAB
+--------------------------------------+
```

2. Plan the composables

Composable	Job	Skills used
Contact (data class)	One contact's data	Data classes (Week 3)
ContactAvatar	A circle with initials, any size	Box, clip, default parameters
FavoriteContactsRow	Horizontal list of favourites	LazyRow, clickable
ContactRow	One contact line with a call button	Row, weight, IconButton
SectionTitle	A letter or section header	Typography, colour tokens
ContactListScreen	The whole screen	Scaffold, TopAppBar, FAB, LazyColumn, item/items, empty state

3. The code

ContactListScreen.kt

```
data class Contact(
    val id: Int,
    val name: String,
    val phone: String,
    val email: String,
    val isFavorite: Boolean = false
)

val sampleContacts = listOf(
    Contact(1, "Anjana Menon", "98470 11111", "anjana@example.com", isFavorite = true),
    Contact(2, "Rahul Nair", "90000 22222", "rahul@example.com"),
    Contact(3, "Meera Thomas", "94470 33333", "meera@example.com", isFavorite = true),
    Contact(4, "John Mathew", "98950 44444", "john@example.com"),
    Contact(5, "Fathima Rasheed", "97450 55555", "fathima@example.com"),
```

```
    Contact(6, "Arun Kumar", "96330 66666", "arun@example.com", isFavorite = true),
    Contact(7, "Divya Pillai", "99610 77777", "divya@example.com"),
    Contact(8, "Sanjay Varma", "95390 88888", "sanjay@example.com")
)

@Composable
fun ContactAvatar(name: String, size: Dp = 48.dp, modifier: Modifier = Modifier) {
    val initials = name.split(" ").take(2).map { it.first() }.joinToString("")
    Box(
        modifier = modifier
            .size(size)
            .clip(CircleShape)
            .background(MaterialTheme.colorScheme.primaryContainer),
        contentAlignment = Alignment.Center
    ) {
        Text(
            text = initials,
            style = MaterialTheme.typography.titleMedium,
            color = MaterialTheme.colorScheme.onPrimaryContainer
        )
    }
}

@Composable
fun FavoriteContactsRow(favorites: List<Contact>, onContactClick: (Contact) -> Unit) {
    LazyRow(
        contentPadding = PaddingValues(horizontal = 16.dp),
        horizontalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        items(favorites, key = { it.id }) { contact ->
            Column(
                horizontalAlignment = Alignment.CenterHorizontally,
                modifier = Modifier
                    .clip(MaterialTheme.shapes.medium)
                    .clickable { onContactClick(contact) }
                    .padding(8.dp)
            ) {
                ContactAvatar(name = contact.name, size = 64.dp)
                Spacer(Modifier.height(4.dp))
                Text(contact.name.substringBefore(" "), style = MaterialTheme.typography.labelMedium)
            }
        }
    }
}

@Composable
fun ContactRow(contact: Contact, onClick: () -> Unit, modifier: Modifier = Modifier) {
    Row(
        modifier = modifier
            .fillMaxWidth()
            .clickable(onClick = onClick)
            .padding(horizontal = 16.dp, vertical = 10.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        ContactAvatar(name = contact.name)
        Spacer(Modifier.width(16.dp))
        Column(modifier = Modifier.weight(1f)) {
            Text(contact.name, style = MaterialTheme.typography.bodyLarge)
            Text(
                text = contact.phone,
                style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )
        }
        IconButton(onClick = { }) {
            Icon(Icons.Default.Call, contentDescription = "Call ${contact.name}")
```

```
        }
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ContactListScreen(
    contacts: List<Contact>,
    onContactClick: (Contact) -> Unit,
    onAddContact: () -> Unit
) {
    val favorites = contacts.filter { it.isFavorite }
    val grouped = contacts.sortedBy { it.name }.groupBy { it.name.first() }

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Contacts") },
                actions = {
                    IconButton(onClick = { }) {
                        Icon(Icons.Default.Search, contentDescription = "Search contacts")
                    }
                }
            )
        },
        floatingActionButton = {
            FloatingActionButton(onClick = onAddContact) {
                Icon(Icons.Default.Add, contentDescription = "Add contact")
            }
        }
    ) { innerPadding ->
        if (contacts.isEmpty()) {
            Box(
                modifier = Modifier.padding(innerPadding).fillMaxSize(),
                contentAlignment = Alignment.Center
            ) {
                Text("No contacts yet. Tap + to add one.")
            }
        } else {
            LazyColumn(
                modifier = Modifier.padding(innerPadding),
                contentPadding = PaddingValues(bottom = 88.dp)   // room for the FAB
            ) {
                if (favorites.isNotEmpty()) {
                    item {
                        SectionTitle("Favourites")
                        FavoriteContactsRow(favorites = favorites, onContactClick = onContactClick)
                        HorizontalDivider(modifier = Modifier.padding(top = 8.dp))
                    }
                }
                grouped.forEach { (letter, contactsForLetter) ->
                    item(key = "header-$letter") {
                        SectionTitle(letter.toString())
                    }
                    items(contactsForLetter, key = { it.id }) { contact ->
                        ContactRow(contact = contact, onClick = { onContactClick(contact) })
                    }
                }
            }
        }
    }
}

@Composable
fun SectionTitle(text: String) {
    Text(
```

```
        text = text,
        style = MaterialTheme.typography.titleSmall,
        color = MaterialTheme.colorScheme.primary,
        modifier = Modifier.padding(start = 16.dp, top = 16.dp, bottom = 8.dp)
    )
}

@Preview(showBackground = true)
@Composable
fun ContactListScreenPreview() {
    ContactListScreen(contacts = sampleContacts, onContactClick = { }, onAddContact = { })
}

@Preview(showBackground = true, name = "Empty")
@Composable
fun ContactListScreenEmptyPreview() {
    ContactListScreen(contacts = emptyList(), onContactClick = { }, onAddContact = { })
}
```

4. Understand the key parts

Code	What it does
contacts.sortedBy { it.name }.groupBy { it.name.first() }	Sorts by name, then groups into a map: 'A' to [Anjana, Arun], 'D' to [Divya]...
grouped.forEach { (letter, list) -> item(...); items(...) }	For each letter, adds a header item and then that letter's contacts
item(key = "header-$letter")	Headers get keys too, with a prefix so they never clash with contact IDs
size: Dp = 48.dp	ContactAvatar is reused at two sizes thanks to a default parameter
contentPadding = PaddingValues(bottom = 88.dp)	Extra space at the end so the FAB doesn't cover the last contact
if (contacts.isEmpty())	The empty state, with its own preview
onContactClick: (Contact) -> Unit	The screen reports clicks. Navigation to a detail screen comes in Week 7

5. Show it in MainActivity

MainActivity.kt (Android only)

```
setContent {
    Week5PracticeTheme {
        val context = LocalContext.current
        ContactListScreen(
            contacts = sampleContacts,
            onContactClick = { contact ->
                Toast.makeText(context, "Clicked ${contact.name}", Toast.LENGTH_SHORT).show()
            },
            onAddContact = { Log.d("Contacts", "Add contact tapped") }
        )
    }
}
```

Here ContactListScreen has its own Scaffold, so MainActivity doesn't need one.

6. Project challenges

- Make the call button open the phone dialer: `context.startActivity(Intent(Intent.ACTION_DIAL, Uri.parse("tel:${contact.phone}")))`.
- Give each avatar a different colour based on the first letter of the name (pick from a small list of theme container colours).
- Add a FilterChip row under the top bar: "All" and "Favourites" (the filtering itself will work once you learn state in Week 6).
- Test the app in **dark mode** and with the **empty list**, then push it to GitHub.

Week 5 checklist

I can...	Done?
Use every arrangement and alignment option in Column and Row	[ ]
Use weight to share space and push elements to the end or bottom	[ ]
Layer content with Box and Modifier.align	[ ]
Build screens with Scaffold, top bars, FABs and a bottom bar	[ ]
Use chips, dividers, badges and progress indicators	[ ]
Customise the theme's colours and fonts, and support dark mode	[ ]
Show lists with LazyColumn, items and stable keys	[ ]
Build LazyRows, grids, sections and empty states	[ ]
Build the Contact List app	[ ]

Quick Quiz

Q1. Why is ContactAvatar a separate composable with a size parameter?

Q2. How does the screen create the alphabetical sections?

Q3. Why does the LazyColumn have bottom contentPadding?

Q4. Why does ContactListScreen take onContactClick instead of handling clicks itself?

Q5. Why are there two previews?

Answer Key

Try every exercise yourself **before** you look here. Your solution may be different and still correct, as long as the output is right!

Day 1: Layouts in Depth

Exercise 1

```
@Composable
fun ChatRow(name: String, time: String, unread: Int) {
    Row(modifier = Modifier.fillMaxWidth().padding(16.dp), verticalAlignment = Alignment.CenterVertically) {
        Box(Modifier.size(48.dp).clip(CircleShape).background(MaterialTheme.colorScheme.primaryContainer))
        Spacer(Modifier.width(12.dp))
        Column(modifier = Modifier.weight(1f)) {
            Text(name, style = MaterialTheme.typography.bodyLarge)
            Text(time, style = MaterialTheme.typography.bodySmall)
        }
        if (unread > 0) {
            Box(
                modifier = Modifier.size(24.dp).clip(CircleShape)
                    .background(MaterialTheme.colorScheme.primary),
                contentAlignment = Alignment.Center
            ) {
                Text("$unread", color = MaterialTheme.colorScheme.onPrimary,
                    style = MaterialTheme.typography.labelSmall)
            }
        }
    }
}
```

Exercise 2

```
@Composable
fun LoginLayout() {
    Column(modifier = Modifier.fillMaxSize().padding(24.dp)) {
        Text("Welcome", style = MaterialTheme.typography.headlineMedium)
        Spacer(Modifier.height(32.dp))
        OutlinedButton(onClick = { }, modifier = Modifier.fillMaxWidth()) { Text("Continue with Google") }
        Spacer(Modifier.height(8.dp))
        OutlinedButton(onClick = { }, modifier = Modifier.fillMaxWidth()) { Text("Continue with phone") }
        Spacer(Modifier.weight(1f))
        Button(onClick = { }, modifier = Modifier.fillMaxWidth()) { Text("Create account") }
    }
}
```

Exercise 3

```
@Composable
fun FourSquares() {
    Column(modifier = Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(8.dp)) {
        Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            Box(Modifier.weight(1f).aspectRatio(1f).background(MaterialTheme.colorScheme.primary))
            Box(Modifier.weight(1f).aspectRatio(1f).background(MaterialTheme.colorScheme.secondary))
        }
        Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            Box(Modifier.weight(1f).aspectRatio(1f).background(MaterialTheme.colorScheme.tertiary))
            Box(Modifier.weight(1f).aspectRatio(1f).background(MaterialTheme.colorScheme.error))
        }
    }
}
```

Exercise 4

```
@Composable
fun DismissibleBanner(text: String, onClose: () -> Unit) {
    Box(
        modifier = Modifier.fillMaxWidth().height(120.dp)
            .background(MaterialTheme.colorScheme.secondaryContainer)
    ) {
        Text(text, modifier = Modifier.align(Alignment.BottomStart).padding(16.dp))
        IconButton(onClick = onClose, modifier = Modifier.align(Alignment.TopEnd)) {
            Icon(Icons.Default.Close, contentDescription = "Close")
        }
    }
}
```

Quiz answers

Q1. verticalArrangement

Q2. SpaceBetween puts no space at the edges; SpaceEvenly makes all gaps equal, including the edges.

Q3. Put Spacer(Modifier.weight(1f)) above it.

Q4. The last one.

Q5. Give that child Modifier.align(...).

Day 2: Material 3 Components & Scaffold

Exercise 1

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ProfileScreen(onBack: () -> Unit) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Profile") },
                navigationIcon = {
                    IconButton(onClick = onBack) {
                        Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription = "Back")
                    }
                },
                actions = {
                    IconButton(onClick = { }) { Icon(Icons.Default.Edit, contentDescription = "Edit") }
                }
            )
        }
    ) { innerPadding ->
        Column(
            modifier = Modifier.padding(innerPadding).padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            OutlinedCard(Modifier.fillMaxWidth()) { Text("Name: Anjana", Modifier.padding(16.dp)) }
            OutlinedCard(Modifier.fillMaxWidth()) { Text("City: Kochi", Modifier.padding(16.dp)) }
        }
    }
}
```

Exercise 2

```
// In NotesScreen's Scaffold, add:
//     bottomBar = { AppBottomBar(selectedTab = 0, onTabSelected = { }) },
@Composable
fun AppBottomBar(selectedTab: Int, onTabSelected: (Int) -> Unit) {
    NavigationBar {
        NavigationBarItem(selected = selectedTab == 0, onClick = { onTabSelected(0) },
            icon = { Icon(Icons.Default.Home, contentDescription = null) }, label = { Text("Home") })
        NavigationBarItem(selected = selectedTab == 1, onClick = { onTabSelected(1) },
            icon = { Icon(Icons.Default.Person, contentDescription = null) }, label = { Text("Profile") })
    }
}
```

Exercise 3

```
@Composable
fun CategoryChips() {
    Row(modifier = Modifier.padding(16.dp), horizontalArrangement = Arrangement.spacedBy(8.dp)) {
        listOf("All", "Work", "Personal", "Shopping").forEachIndexed { index, label ->
            FilterChip(selected = index == 0, onClick = { }, label = { Text(label) })
        }
    }
}
```

Exercise 4

```
@Composable
fun DownloadCard() {
    Card(modifier = Modifier.fillMaxWidth().padding(16.dp)) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text("Downloading lesson.mp4", style = MaterialTheme.typography.titleMedium)
            Spacer(Modifier.height(12.dp))
            Row(verticalAlignment = Alignment.CenterVertically) {
                LinearProgressIndicator(progress = { 0.35f }, modifier = Modifier.weight(1f))
                Spacer(Modifier.width(12.dp))
                Text("35%")
            }
        }
    }
}
```

Quiz answers

Q1. It places the top bar, bottom bar, FAB, snackbar and content in the right positions.

Q2. So the content isn't hidden behind the top and bottom bars.

Q3. FilterChip

Q4. BadgedBox with badge = { Badge { Text("4") } }

Q5. Changing the text needs state, which is in Week 6.

Day 3: Themes, Colours & Fonts

Exercise 1

```
// In Theme.kt:
// private val LightColorScheme = lightColorScheme(primary = Color(0xFF00696B))
// private val DarkColorScheme = darkColorScheme(primary = Color(0xFF4CDADB))
// Then run the app in light and dark mode.
@Composable
fun PrimaryColorCheck() {
    Button(onClick = { }) { Text("This button uses the new primary colour") }
}
```

Exercise 2

```
// Replace fixed colours with tokens, for example:
@Composable
fun CardTitle(text: String) {
    Text(text, color = MaterialTheme.colorScheme.onPrimaryContainer,
        style = MaterialTheme.typography.headlineMedium)
}
```

Exercise 3

```
// Android only: see the Type.kt example in section 4 (FontFamily + Typography).
@Composable
fun FontCheck() {
    Text("Title in the new font", style = MaterialTheme.typography.titleLarge)
}
```

Exercise 4

```
@Composable
fun InfoChip(text: String) {
    Text(
        text = text,
        color = MaterialTheme.colorScheme.onSecondaryContainer,
        modifier = Modifier
            .background(MaterialTheme.colorScheme.secondaryContainer, MaterialTheme.shapes.small)
            .padding(horizontal = 12.dp, vertical = 6.dp)
    )
}

@Composable
fun InfoChipBothThemes() {
    Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
        MaterialTheme(colorScheme = lightColorScheme()) { Surface { InfoChip("Light") } }
        MaterialTheme(colorScheme = darkColorScheme()) { Surface { InfoChip("Dark") } }
    }
}
```

Quiz answers

Q1. Theme.kt

Q2. It returns true when the phone is in dark mode.

Q3. onPrimaryContainer

Q4. res/font

Q5. On Android 12+, a colour scheme generated from the user's wallpaper.

Day 4: Lists with LazyColumn

Exercise 1

```
data class Message(val id: Int, val sender: String, val preview: String, val time: String)

val sampleMessages = (1..10).map { Message(it, "Sender $it", "Message preview number $it", "10:0$it") }

@Composable
fun MessageList(messages: List<Message>) {
    LazyColumn {
        items(messages, key = { it.id }) { message ->
            Column(modifier = Modifier.fillMaxWidth().padding(16.dp)) {
                Text(message.sender, style = MaterialTheme.typography.titleSmall)
                Text(message.preview, maxLines = 1, overflow = TextOverflow.Ellipsis)
            }
            HorizontalDivider()
        }
    }
}
```

Exercise 2

```
data class Message(val id: Int, val sender: String, val preview: String, val isUnread: Boolean)

@Composable
fun MessageItem(message: Message) {
    Column(modifier = Modifier.fillMaxWidth().padding(16.dp)) {
        Text(
            text = message.sender,
            fontWeight = if (message.isUnread) FontWeight.Bold else FontWeight.Normal
        )
        Text(message.preview, maxLines = 1, overflow = TextOverflow.Ellipsis)
    }
}
```

Exercise 3

```
@Composable
fun ContactCards(contacts: List<Contact>, onContactClick: (Contact) -> Unit) {
    LazyColumn(
        contentPadding = PaddingValues(16.dp),
        verticalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        items(contacts, key = { it.id }) { contact ->
            Card(onClick = { onContactClick(contact) }, modifier = Modifier.fillMaxWidth()) {
                Text(contact.name, modifier = Modifier.padding(16.dp))
            }
        }
    }
}
```

Exercise 4

```
@Composable
fun TopMovies(movies: List<String>) {
    LazyColumn {
        itemsIndexed(movies, key = { _, title -> title }) { index, title ->
            Text("${index + 1}. $title", style = MaterialTheme.typography.bodyLarge,
                modifier = Modifier.padding(16.dp))
        }
    }
}
```

Quiz answers

Q1. LazyColumn only creates the visible items, so it's fast for long lists and scrolls by itself.

Q2. It gives each item a unique, stable identity, so scrolling and animations stay correct.

Q3. The app crashes.

Q4. So the caller decides what happens on a click, which keeps the list reusable.

Q5. contentPadding adds space inside the scrolling area, so items scroll to the edge; padding cuts off items at the padding.

Day 5: LazyRow, Grids & List Patterns

Exercise 1

```
@Composable
fun CategoryChipRow() {
    val categories = listOf("All", "Food", "Travel", "Bills", "Shopping", "Health")
    LazyRow(
        contentPadding = PaddingValues(horizontal = 16.dp),
        horizontalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        itemsIndexed(categories, key = { _, c -> c }) { index, category ->
            FilterChip(selected = index == 0, onClick = { }, label = { Text(category) })
        }
    }
}
```

Exercise 2

```
data class Product(val id: Int, val name: String, val price: Int)

@Composable
fun ProductGrid(products: List<Product>) {
    LazyVerticalGrid(
        columns = GridCells.Adaptive(150.dp),
        contentPadding = PaddingValues(16.dp),
        horizontalArrangement = Arrangement.spacedBy(12.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        items(products, key = { it.id }) { product ->
            Card {
                Column(modifier = Modifier.padding(12.dp)) {
                    Box(Modifier.fillMaxWidth().aspectRatio(1f)
                        .background(MaterialTheme.colorScheme.surfaceVariant))
                    Spacer(Modifier.height(8.dp))
                    Text(product.name, style = MaterialTheme.typography.titleSmall)
                    Text("Rs ${product.price}")
                }
            }
        }
    }
}
```

Exercise 3

```
@Composable
fun CountryList(countries: List<String>) {
    val grouped = countries.sorted().groupBy { it.first() }
    LazyColumn {
        grouped.forEach { (letter, names) ->
            item(key = "header-$letter") {
                Text(letter.toString(), style = MaterialTheme.typography.titleSmall,
                    color = MaterialTheme.colorScheme.primary, modifier = Modifier.padding(16.dp))
            }
            items(names, key = { it }) { name ->
                Text(name, modifier = Modifier.padding(horizontal = 16.dp, vertical = 8.dp))
            }
        }
    }
}
```

Exercise 4

```
@Composable
fun ContactListWithEmpty(contacts: List<Contact>) {
    if (contacts.isEmpty()) {
        Box(Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
            Text("No contacts yet")
        }
    } else {
        LazyColumn {
            items(contacts, key = { it.id }) { contact ->
                Text(contact.name, modifier = Modifier.padding(16.dp))
            }
        }
    }
}
```

Quiz answers

Q1. LazyRow

Q2. Fixed always uses a set number of columns; Adaptive fits as many columns of a minimum width as possible.

Q3. item { SectionHeader(...) }

Q4. To keep keys unique when the same item appears in more than one section.

Q5. Yes, because they scroll in different directions.

Day 6: Weekly Project: Contact List App

Quiz answers

Q1. It's reused in two places at different sizes.

Q2. It sorts and groups the contacts with groupBy, then adds a header item and the items for each letter.

Q3. So the floating action button doesn't cover the last contact.

Q4. So the caller (and later the navigation) decides what happens, which keeps the screen reusable and previewable.

Q5. To check both the normal state and the empty state.

Next: When you finish this week, tell your mentor and ask for the **Week 6** material. Great work, Anjana!